

CSE 511: NoSQL Project Report

Name: Sejal Patil | ASU ID: 1239407081 | email id: sapati11@asu.edu

Reflection:

In this project, I implemented two Python functions to query business data stored in an UnQLite NoSQL database. The main goal was to create two functions that could search through business data - one by city name and another by location using coordinates and then output the query results into two text files named output_city.txt and output_loc.txt simultaneously.

Implemented Functions:

1.FindBusinessBasedOnCity: This function accepts three parameters: cityToSearch (string), saveLocation1 (file path), and collection (database collection). I used the collection's filter method with a lambda function to pull out only the businesses that matched the city I was looking for. Then I had to format each result with dollar signs as delimiters between the name, address, city, and state. This formatted data is written to the specified output file with each business on a separate line.

```
def FindBusinessBasedOnCity(cityToSearch, saveLocation1, collection):
    results = collection.filter(lambda record: record['city'] == cityToSearch)

    with open(saveLocation1, 'w') as f:
        for business in results:
            line = f"{business['name']}${business['full_address']}${business['city']}${business['state']}\n"
            f.write(line)
```

2.FindBusinessBasedOnLocation: This function finds businesses within a certain distance from given location, filtered by category. It then saves the matching business names to a text file.

Distance Calculation Function:

Firstly, I created helper functions for degree-to-radian conversion and then implemented the Haversine formula to calculate distance between two points using latitude and longitude coordinates and convert them to actual miles.

Main code logic:

This function accepts five parameters: categoriesToSearch (list), myLocation (coordinate pair), maxDistance (float), saveLocation2 (file path), and collection (database collection). The implementation retrieves all businesses from the collection, for each business, it first validates the existence of latitude, longitude and categories to prevent runtime errors. If the

distance(cal_distance) is within the specified threshold, the function checks if any of the business's categories match the search criteria using OR logic. When a match is found, the business name is added to the results list, and the loop breaks to prevent duplicate entries. Finally, all matching business names are written to the output file, one per line.

```
def FindBusinessBasedOnLocation(categoriesToSearch, myLocation, maxDistance, saveLocation2, collection):
    def toRadians(degrees):
        return degrees * math.pi / 180

    def cal_distance(lat1, lon1, lat2, lon2):
        R = 3959
        phi1 = toRadians(lat1)
        phi2 = toRadians(lat2)
        delta_phi = toRadians(lat2 - lat1)
        delta_lambda = toRadians(lon2 - lon1)
        a = (math.sin(delta_phi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(delta_lambda / 2) ** 2)
        c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
        d = R * c
        return d

    my_lat, my_lon = myLocation
    all_businesses = collection.all()
    matching_businesses = []

    for business in all_businesses:
        if 'latitude' not in business or 'longitude' not in business or 'categories' not in business:
            continue

        distance = cal_distance(my_lat, my_lon, business['latitude'], business['longitude'])

        if distance <= maxDistance:
            for category in categoriesToSearch:
                if category in business['categories']:
                    matching_businesses.append(business['name'])
                    break

    with open(saveLocation2, 'w') as f:
        for business_name in matching_businesses:
            f.write(f"{business_name}\n")
```

Lessons Learned:

- Understood fundamental differences between document-based (UnQLite) and relational database systems related to data storage and query.
- Gained a deeper understanding of how theoretical concepts, like the Haversine formula, can be applied to solve real-world problems.
- Learned to use test cases effectively to validate function correctness.
- Practiced reading from databases and writing formatted output to text files with specific delimiters.

Output:

Below are the screenshots of my txt output files.

File Edit View Settings Help

```
1 |VinciTorio's Restaurant$1835 E Elliot Rd, Ste C109, Tempe, AZ 85284$Tempe$AZ
2 |Salt Creek Home$1725 W Ruby Dr, Tempe, AZ 85284$Tempe$AZ
3 |P.croissants$7520 S Rural Rd, Tempe, AZ 85283$Tempe$AZ
4 |
```

File Edit View Settings Help

```
1 |VinciTorio's Restaurant
2 |
```

Results:

Test Case 1: FindBusinessBasedOnCity

Input:

- City: 'Tempe'

Output:

Correct! You FindBusinessByCity function passes these test cases. This does not cover all possible test edge cases, however, so make sure that your function covers them before submitting!

Test Case 2: FindBusinessBasedOnLocation

Input:

- Categories: ['Buffets']
- Location: [33.3482589, -111.9088346]
- Maximum Distance: 10 miles

Output:

Correct! Your FindBusinessBasedOnLocation function passes these test cases. This does not cover all possible edge cases, so make sure your function does before submitting.